Claude Code Prompt — PrecisionBridge AI MVP

> Before pasting: copy `EXL_Personalized_Medicine_PRD.docx` into the new repo as `docs/PRD.docx` (or convert it to `docs/PRD.md`). The prompt below tells Claude Code to treat it as the source of truth.

You are acting as a **principal engineer and software architect** building the MVP of **PrecisionBridge AI**, a clinician-supervised, two-wing (Patient Wing + Research Wing) decision-support and research-hypothesis prototype that runs on **synthetic data only**. The full Product Requirements Document is in `docs/PRD.docx` (or `docs/PRD.md`). Read it completely before writing any code. Where this prompt and the PRD disagree, the **safety invariants** below win, then the PRD, then the rest of this prompt.

Deployment target: code in a GitHub repo, hosted on **Vercel's free (Hobby) plan**, with zero paid services. Every architectural choice must fit that constraint.

0. How to work

1. Start in **plan mode**. Read the PRD, then produce: an architecture summary, the repo layout, the data model, a mapping of every P0 feature (F01–F17) and test (T01–T18) to the files/tests that implement it, and a phased plan. Wait for my approval before writing code.
2. Build **one complete vertical slice first** (hypertension, golden path end to end), then port diabetes and depression as **data-only module packs**. Do not build breadth before the slice works.
3. Work in small phases. At the end of each phase: run lint, typecheck and tests; commit with a conventional-commit message; tell me what's done and what's next. **Ask before pushing to GitHub or changing anything in Vercel.**
4. Never invent clinical facts. Where the PRD needs clinical content, use clearly labeled illustrative content (see §5).
5. If you hit a genuine ambiguity, list your assumption in `docs/DECISIONS.md` (lightweight ADR log) and continue.

1. Non-negotiable safety invariants (enforce in code AND tests)

These are architectural, not UI conventions. Each must have at least one automated test that fails if it's broken.

- **I1 — Safety before everything.** Every check-in is evaluated by a deterministic rule engine *before* any option comparison, conversational text or research processing.
- **I2 — Fail closed.** Missing critical data can never produce ROUTINE. Any exception, timeout or invalid module in the safety path yields `UNABLE_TO_ASSESS`. URGENT and UNABLE_TO_ASSESS block option comparison.

- **I3 — Structured beats free text.** A positive structured red-flag answer cannot be down-classified by free text or any model output (T06).
- **I4 — Zero autonomous medication changes.** The system never generates doses, never auto-executes a plan, never shows option rankings to patients. Medication-related plans stay `draft` until a clinician signs.
- **I5 — Signed records are immutable.** A plan change creates a new version with signer, timestamp and reason; prior versions are never mutated (T08).
- **I6 — No uncited option cards.** Every option card carries module id/version, guideline source id, review date and rationale, or it is withheld with "Insufficient evidence — manual review required" (T14).
- **I7 — PGx abstains by default.** PGx annotations appear only for a valid, documented genotype result AND a drug-gene pair listed in the module's `drug_gene_evidence`. Otherwise: "No actionable PGx conclusion" (T07).
- **I8 — Research isolation.** Researchers never receive direct identifiers or patient-level routes (403, no payload — T10). Records without research consent are excluded (T09). Cells with n < 10 are suppressed (configurable) and cannot be promoted to hypotheses (T11).
- **I9 — No research → care path.** Nothing in the Research Wing can write to modules, option comparisons or care plans. Enforce this with module boundaries (lint rule or import restrictions) plus a test.
- **I10 — Everything audited.** Consent changes, rule evaluations (with rule ids + module version), alerts, comparisons, clinician decisions, research access/denials and module activations write an append-only `AuditEvent`. No PHI-like payloads in logs.
- **I11 — Honest escalation.** URGENT shows pre-approved emergency guidance synchronously and creates an alert; alerts show "not yet acknowledged" until a clinician actually acknowledges them.
- **I12 — Watermark.** Every screen and export shows **"SYNTHETIC DATA — NOT FOR CLINICAL USE."**

2. Tech stack (chosen for Vercel Hobby + zero cost)

- **Next.js (App Router) + TypeScript (strict)**, single repo, deployed to Vercel. Use Route Handlers for the API (`/app/api/...`) mirroring the PRD's API shortlist; use Server Components / Server Actions for the UI where it simplifies things, but keep all business logic in framework-free modules under `src/core`.
- **Database:** Postgres on **Neon free tier** (added through the Vercel Marketplace, which injects `DATABASE_URL`). ORM: **Drizzle** with migrations checked in. Use the Neon serverless driver. Local dev can point at a Neon branch or a local Postgres via Docker — document both.
- **Validation: Zod** for every API input/output, module manifest and agent output type.

- **Auth/RBAC:** no paid auth provider. Seeded demo users per role (patient, clinician, researcher, clinical_admin, platform_admin) with a "sign in as demo user" picker. Session = signed, httpOnly cookie (`jose`, secret from env). Enforce RBAC **server-side in every route handler** via a single `authorize(user, action, resource)` policy function; middleware is only a convenience layer, never the sole check. Clinicians see only assigned patients.
- **UI:** Tailwind CSS + shadcn/ui; Recharts for trends and cohort charts. Accessible: keyboard-operable forms, status labels never color-only.
- **LLM: not required for P0.** The MVP must run fully deterministically with no API key. If time permits, F21 (paraphrasing approved questions) goes behind `ENABLE_LLM=false` by default, with the structured question/answer always preserved.
- **Testing:** Vitest (unit, rule engine, policy, contract, API authorization tests); Playwright for 2–3 golden-path E2E flows.
- **CI:** GitHub Actions on PR/push: install, lint, typecheck, Vitest. Vercel's Git integration handles preview and production deploys.

Free-tier design rules: keep every request fast and short-lived (no long-running jobs, no background workers, no websockets); compute cohort aggregates with SQL; seed via a local script, not at request time; keep the bundle lean.

3. Architecture

```
src/
  core/                    # pure TypeScript, no Next.js imports
    modules/               # manifest schema, loader, validator (F15, T18)
    safety/                # deterministic rule engine (F05, I1-I3)
    comparison/            # option-comparison pipeline (F08, F09, I6, I7)
    plans/                 # decisions, versioned care plans (F10, I4, I5)
    research/              # governance gate, cohorts, suppression,
hypotheses (F12-F14, F19, I8)
    trials/                # P1 criterion-level pre-screen (F18)
    policy/                # RBAC authorize() (F01)
    audit/                 # append-only audit writer (F16)
    types/                 # Zod schemas = typed agent outputs
  db/                      # Drizzle schema, migrations, queries
  app/                     # Next.js routes: (patient) (clinician) (research)
(admin) + api/
modules/                   # disease packs as JSON: hypertension, t2d,
depression
fixtures/                  # synthetic seed generator + separate expected-
label files
tests/                     # unit, contract, api-authz, e2e
docs/                      # PRD, DECISIONS.md, LIMITATIONS.md,
DEMO_SCRIPT.md
```

- "Agents" in the PRD are implemented as **orchestrated, typed services**, not autonomous bots. Each returns a Zod-validated output with `source`, `module_version`, `limitations`, `timestamp`.
- **Patient Wing orchestrator:** validate check-in → safety evaluation → (if allowed) profile synthesis → option comparison draft for clinician → follow-up against approved plan.
- **Research Wing orchestrator:** consent/purpose gate → de-identified research view (a DB view or materialized table with surrogate IDs, no names/contact data) → cohort definition → aggregates with suppression → hypothesis queue (P1).
- **Disease-agnostic core (G7/F15):** no `if (moduleId === ...)` anywhere in `src/core`. Add a test that greps core for module ids and fails if found.

4. Data model

Implement the PRD §11 entities in Drizzle: Patient, PatientConsent, CheckIn, Observation, MedicationExposure, Condition, Allergy, GenotypeEvidence, RiskEvaluation, Alert, GuidelineSource, DiseaseModuleVersion, OptionComparison, ClinicianDecision, ApprovedCarePlan, ResearchCohortDefinition, ResearchAggregate, Hypothesis, TrialSnapshot, AuditEvent, plus User/Role/Assignment.

Every clinical datum carries `value, unit, observed_at, source (patient|clinician|lab|synthetic), verification_status, null_reason (not_asked|unknown|not_available|not_applicable)`. Never impute missing as normal. Flag unit/date conflicts and exclude them from computation (T17). AuditEvent and signed plan versions are append-only (no update/delete paths in code; add a test).

5. Disease modules and clinical content

- Define the manifest schema from PRD §9 in Zod. Ship three JSON packs. Modules are immutable per version; activation requires schema validation, a named reviewer and passing the safety regression subset (T18).
- **No qualified clinician has reviewed this content**, so per PRD §17 and §19: use **drug classes, not specific drugs, never doses**, and mark every card `content_status: "illustrative — not clinically reviewed"`. Guideline sources reference the documents listed in PRD §20 by id/title/date only; do not fabricate recommendation text or page numbers — use short neutral summaries labeled as illustrative.
- Red-flag rules are data (JSON condition trees evaluated by the engine), each with an id, rationale and `clinically_approved: false (illustrative)`. Do not invent universal patient-facing thresholds; put threshold values in clearly labeled config.
- Depression module: PHQ-9-style structured item set with a **direct self-harm question** that, if positive, triggers URGENT and immediate help guidance — no conversational therapy.
- PGx: only the depression module includes illustrative `drug_gene_evidence` (CYP2D6/CYP2C19/CYP2B6 per PRD §9). Hypertension and diabetes have none.
- Emergency copy lives in one admin-controlled config, targeted at India by default (general emergency 112; mental-health line Tele-MANAS 14416), with a visible note "verify numbers before any demo."

6. Synthetic data (F17)

Deterministic seeded generator (fixed seed, e.g. `seedrandom` + `@faker-js/faker` with seed): 300 patients (100/condition), 4–8 check-ins each, care-plan snapshots, consent variations, assignments. Guarantee the PRD §10 coverage matrix per module (≥10 routine, ≥10 high-risk, ≥10 missing/contradictory, plus contraindication, PGx, consent, role-pair, spurious-correlation and small-cell cases). Include named storyboard patients **DM-042** and **DEP-017** exactly as described in PRD §18. Store expected labels in `fixtures/expected/` separately from data. Provide `npm run db:seed` and `npm run db:reset`, plus an admin-only "reset demo data" action.

7. UI surfaces

- **Patient app:** onboarding with consent (clinical and research separate), profile with completeness indicator and provenance, disease-aware check-in, status result, approved plan and follow-up only. Never shows option cards.

- **Clinician cockpit:** caseload queue (urgent / prompt / pending), patient summary, trend timeline with missingness, alert details + acknowledge, "what changed" panel, side-by-side option cards with evidence drawer, decision composer (approve_plan, request_more_data, reject_candidate, record_alternative, mark_alert_reviewed, refer_to_human_service), plan version history.
- **Research workbench:** cohort builder with explicit endpoint and horizon, subgroup n/N, missingness, suppression markers, caveat banners, saved reproducible filter definitions; P1 hypothesis queue (dismiss / needs_evidence / request_scientific_review) labeled "demonstration-only; not evidence of efficacy"; P1 trial pre-screen at criterion level from a dated mock registry snapshot.
- **Admin console:** module list/validate/activate/rollback, emergency copy, audit log viewer.
- Persistent watermark banner, role indicator and "Demo only" banner on every page.

8. Testing and Definition of Done

- Implement **T01–T18 from PRD §15** as named Vitest tests (`T01_self_harm_urgent.test.ts`, etc.), plus negative RBAC tests for every API route and contract tests that load all three modules through the same engines.
- Playwright: (a) DM-042 storyboard scenes 1–3, (b) DEP-017 urgent path, (c) researcher cohort + hypothesis dismissal.
- A `npm run kpi` script that runs the fixture set and prints the PRD §13 KPIs (critical test recall, autonomous changes = 0, evidence coverage, abstention, audit capture, research isolation, module portability 3/3, reproducibility) as **measured** values beside targets.
- **Done** = all P0 features working on the deployed Vercel URL, all T01–T18 green in CI, KPI script output saved to `docs/KPI_RESULTS.md`, and `docs/DEMO_SCRIPT.md` + `docs/LIMITATIONS.md` written (no clinical or regulatory claims).

9. Phases

1. **Foundation:** repo scaffold, CI, Drizzle schema + migrations, Zod types, module manifest schema + validator, policy/RBAC, audit writer, demo sign-in. Tests for I8/I10/T18.
2. **Patient Wing slice (hypertension):** profile, check-in, safety engine, option comparison, PGx abstention. Tests T03–T07, T14–T17.
3. **Clinician cockpit:** queue, timeline, alerts, decisions, versioned plans, patient follow-up view. Test T08.
4. **Module port:** diabetes + depression packs with zero core changes. Tests T01, T02, portability test.
5. **Research Wing:** de-identified view, consent gate, cohorts, suppression, dashboard. Tests T09–T12.
6. **Seed + E2E + KPI:** full generator, Playwright flows, KPI script.

7. **P1 (only if all P0 is green):** trial pre-screen (T13), hypothesis queue, exports with watermark. Cut P1 first if behind.
8. **Deploy:** see §10.

10. GitHub + Vercel deployment

When I approve deployment:

1. Ensure `.env.example` lists `DATABASE_URL`, `SESSION_SECRET`, `ENABLE_LLM`, `SUPPRESSION_THRESHOLD`; `.env*` is git-ignored; no secrets in code.
2. Initialize git, create the GitHub repo (use `gh` CLI if available; otherwise give me the exact commands), push `main`.
3. Give me step-by-step instructions to: import the repo in Vercel (Hobby), add **Neon** from the Vercel Marketplace (free plan) so `DATABASE_URL` is injected, set `SESSION_SECRET`, run migrations (`npm run db:migrate`) and the seed against that database from my machine, and redeploy.
4. Smoke-test the production URL: sign in as each role, run the DM-042 and DEP-017 scenes, confirm a researcher gets 403 on a patient URL.
5. Add the live URL and a short architecture diagram (Mermaid) to `README.md`.

Begin now with step 0: read the PRD and return your plan for approval.